

Human vs. Machine Text Classification

Kaggle Team: **The Optimizers**

Vidhi Marathe (28902) Anmol Gupta (27257) Nunavath Saidulu (26534)

September 2026 – Public leaderboard accuracy: **0.9413**

1 Problem and Data

We classify documents as **A** (human-written) or **B** (machine-generated), given only anonymised integer token sequences drawn from a vocabulary of 18,438 IDs. Because the token-to-word mapping is withheld, word embeddings and pretrained language models are unusable; every signal must be derived from token statistics and sequence structure. The training set has 10,536 labelled documents (3,699 A / 6,837 B – a 35.1%/64.9% split), and the test set has 3,000 unlabelled documents. Human documents are shorter on average (mean 137.5 tokens) than machine documents (mean 213.2 tokens), and Token 0 (the reserved rare-word marker) is used at different rates by the two classes (Figure 1); these two cues alone are informative but far from sufficient, which motivated a much larger, dedicated feature set described in Section 3.

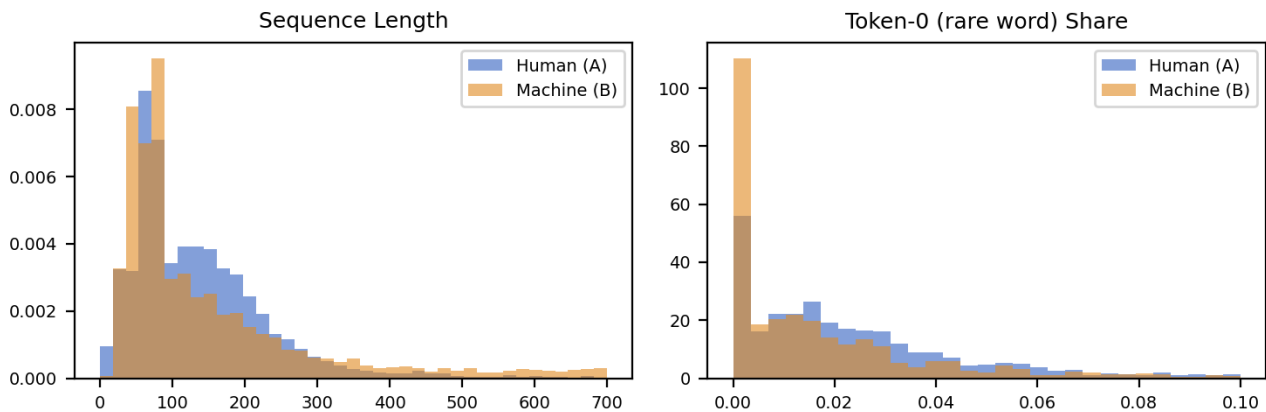


Figure 1: Class-conditional distributions of sequence length and Token-0 share (training set).

2 Preprocessing and Validation

No external data or de-anonymisation of tokens was attempted, in line with the rules. Because of the 1:1.85 class imbalance, all splits used `StratifiedKfold`/stratified holdouts. Every feature that depends on the label – TF-IDF vocabularies, n -gram Markov tables, and scalar statistics – was fit *strictly inside* each training fold and only applied (never re-fit) to the corresponding validation fold and to the test set, to avoid leakage into the reported cross-validation (CV) numbers. A single script-level assertion guard was added that raises an error if any out-of-fold (OOF) or auxiliary cache file is accidentally loaded during a “certified” run, so that reported CV accuracy cannot be silently inflated by stale artefacts. We used 5-fold CV throughout, since no explicit validation set was released.

3 Final Approach: Multi-Component Stacking + Multi-Seed Bagging

Our final, submitted pipeline (`submission_candidate_5seed_bagged.csv`) is a two-level stack, repeated over 5 random seeds and averaged.

Feature blocks. For every document we compute: (i) *34 deterministic statistics* – sequence length, type-token ratio, Shannon entropy, Token-0 rate, adjacent-token repetition, zlib compression ratio, inter-token gap statistics (mean/Fano-factor/CV), a Heaps’-law exponent, early-vs-late entropy drift, and multi-window (16/32/64/128-token) rolling-entropy mean/std; (ii) *41 static sequence-geometry features* – Rényi-2 and Simpson/Gini diversity, gzip compression, run-length statistics, moments and quantiles of the raw token IDs, positional type-token drift, and FFT-based spectral entropy/centroid of the token sequence treated as a numeric series; together these 75 features (“Comb75”) are label-independent and computed once. (iii) *A positional n -gram language model*: class-conditional (A vs. B) unigram/bigram/trigram counts are tabulated in 10 relative-position bins on the training fold only, Laplace-smoothed, and interpolated across position-specific, global, and unigram-backoff estimates; from this we derive per-document log-likelihood-ratio summaries under the A-model vs. the B-model (13 positional-trigram features, 16 bigram+trigram log-likelihood features, and 36 features from sliding 8/16/32/64-token log-likelihood windows). (iv) Character(1–5)-gram and word(1–5)-gram TF-IDF, and a 32-component TruncatedSVD of the character TF-IDF.

Base learners (8 per seed). LinearSVC ($C=14$) on char-TF-IDF+Comb75; LinearSVC ($C=10$) on word-TF-IDF+Comb75;

three more LinearSVC ($C=14$) variants on char-TF-IDF plus, respectively, the positional-trigram, the bigram+trigram, and the windowed Markov features; a RidgeClassifier ($\alpha=10$) on the concatenation of char- and word-TF-IDF with the positional-trigram features; a HistGradientBoostingClassifier (250 trees, depth 5) on SVD(32)+Comb75; and a PassiveAggressiveClassifier on char-TF-IDF+Comb75. All 8 are fit in-fold (5-fold CV) to build an OOF probability matrix, and a **LogisticRegression** ($C=5$) **meta-learner** is fit on that $10,536 \times 8$ matrix.

Bagging. The entire procedure (fold splits, Markov tables, TF-IDF vocabularies, all 8 base learners, and the meta-learner) is re-run independently for 5 seeds (42, 123, 2026, 3407, 7777); the resulting test probabilities are averaged and thresholded at 0.5. Averaging probabilities rather than hard votes was chosen because pairwise inter-seed correlation of the raw probabilities was consistently very high, so mean-probability, median, logit-averaging and majority-vote aggregation nearly always agreed – bagging mainly damps the few borderline cases where individual seeds disagree, rather than changing the model qualitatively.

4 Alternative Methods Considered

Before arriving at the pipeline above, we tried several progressively more complex strategies (Table 1):

- **Single linear baseline** – LinearSVC on char(1–4)-gram TF-IDF plus the 34 deterministic features gave $\approx 89.7\%$ on a held-out 80/20 split. This confirmed the token-statistics signal was strong but not sufficient alone.
- **7-component stack (single seed)** – the same idea as our final model but without the Markov language-model block and without multi-seed bagging; this reached a certified 5-fold OOF accuracy of **92.52%** and was our first Kaggle submission.
- **Consensus/voting ensembles across teammates’ models** (unanimous vote, simple majority, team-weighted consensus, conservative 5-vote, and score-weighted consensus variants) – we combined predictions from each team member’s independently trained model by unanimity, simple majority, and leaderboard-score-weighted voting. These reached 0.9060–0.9400 on the public leaderboard: competitive, but hard voting over a small number of heterogeneous models discards the confidence information a soft, learned meta-learner can exploit, and never matched the probability-averaged stack.
- **8-component stack + Markov features, single seed** – adding the positional-trigram/bigram language model and PassiveAggressive base learner to the stack, still with one seed.
- **8-component stack + Markov + 5-seed bagging (final)** – reached **0.9413** on the public leaderboard, our highest and final scored submission.

The general trend – richer, leakage-safe feature blocks plus stacking beat any single model, and averaging across random seeds gave a further, smaller but consistent gain – matches the intuition that both feature diversity and variance reduction contribute, but along different axes, and is why we kept both mechanisms in the final pipeline rather than only one.

5 Feature Engineering: A Controlled Experiment (Bonus Component)

While integrating the Markov language-model block into the base learners we found that naively concatenating the raw, *un-normalised* positional-trigram log-likelihood features onto the sparse TF-IDF matrix for a LinearSVC caused holdout accuracy to **collapse from 89.71% to 71.44%** on an identical 80/20 split (Figure 2). Inspecting the raw feature revealed the cause: the cumulative log-likelihood-ratio feature (summed over an entire document, not length-normalised) ranges from about $-2,700$ to $+1,400$ with a standard deviation near 475, whereas the other 12 Markov summary features and the 34 deterministic features have standard deviations between roughly 0.2 and 50. Standard scaling equalises *variance* but not the heavy-tailed shape of this distribution, so a handful of extreme documents dominate the margin computed by the linear SVM once it is mixed with thousands of small, sparse TF-IDF weights – the same sparse/dense scale-conflict phenomenon reported for stylometric features in related work. Our fix, already built into the production pipeline (Section 3), is to keep only length-normalised and windowed/positional summaries (mean, std, min/max, sign-fraction) rather than raw un-normalised sums, and to let the tree-based (HistGradientBoosting) and the probability-level meta-learner absorb any residual scale differences, since both are far less sensitive to monotonic or heavy-tailed rescaling than a Euclidean-margin linear model. This is a genuine, controlled, same-protocol (identical stratified split, identical base model, identical hyperparameters) before/after comparison, not a difference attributable to a larger model.

6 Results

Table 1 summarises our submission history. The final 5-seed bagged stack is the strongest of all eleven submissions we made and is the one selected for scoring.

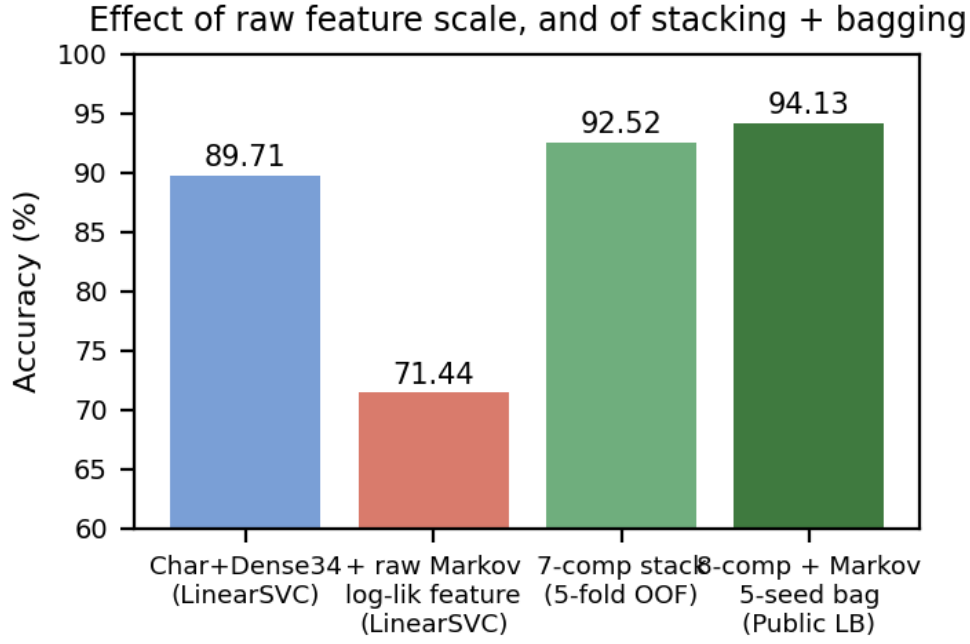


Figure 2: Left two bars: controlled ablation showing the raw-scale Markov feature collapsing a single LinearSVC (identical 80/20 split, seed 42). Right two bars: progression of our staged approaches.

Submission	Method	Public Acc.
5seed.bagged	8-comp. stack + Markov LM, 5 seeds	0.9413
v8.team.weighted.consensus	Score-weighted vote (team models)	0.9400
v4.unanimous	Unanimous vote (team models)	0.9400
v9.conservative.5vote	Conservative 5-vote	0.9346
v10.score.weighted.consensus	Score-weighted consensus	0.9320
v5.consensus	Simple consensus vote	0.9306
submission	7-comp. stack (single seed)	0.9300
blend.v1	Early prediction blend	0.9226
v3	Single-model iteration	0.9073
v6.prior1948	Prior-recalibrated threshold	0.9060
v2	Single-model baseline	0.9020

Table 1: All submission accuracies (Kaggle public leaderboard, 50% of test set).

Internally, the certified 5-fold OOF accuracy of the (simpler) single-seed 7-component stack was 92.52%, and adding the Markov block plus 5-seed bagging lifted the public leaderboard score to 94.13%; the gap between the two numbers is expected since OOF/CV accuracy and the public 50%-of-test leaderboard score are measured on different data.

7 Limitations

Because every feature is derived purely from anonymised token IDs and their statistics, our model has learned the specific stylistic and repetition habits of the particular generator(s) used to build this dataset rather than any semantic notion of “machine-like” writing; if the underlying generator were changed, the Markov tables and TF-IDF vocabularies would need to be refit. The heavy stacking/bagging pipeline is also computationally expensive (8 models $\times$ 5 folds $\times$ 5 seeds) relative to its marginal accuracy gain over the single-seed stack, which is a trade-off worth flagging for future iterations that must run under tighter time budgets.

References: Pedregosa et al., “Scikit-learn: Machine Learning in Python,” JMLR 12, 2011; Jawahar, Abdul-Mageed & Lakshmanan, “Automatic detection of machine generated text: A critical survey,” COLING 2020; Micci-Barreca, “A preprocessing scheme for high-cardinality categorical attributes,” ACM SIGKDD Explorations 3(1), 2001 (motivating in-fold Markov table fitting).